Job-shop disruption assistant

The complete guide: what it is, what was built, how it works, how to run it, and how to explain it

What	An AI assistant that helps a factory planner re-plan after a disruption, where the AI can try changes but never apply them
Repository	github.com/roopchandrika/jobshop-agent (public, MIT licence, synthetic data only)
State at writing	Phases 0 to 15 built and pushed; 1424 automated tests pass; CI green on every push; container built and run
Purpose	A portfolio project for learning AI engineering: every concept an AI or agentic-AI engineer is expected to know, built and tested for real
Headline results	Against a real model (claude-sonnet-5-5): 0 of 56 live red-team runs succeeded (12 attacks) and the live plan never changed; 32 of 35 judged scenarios passed every check. Against a worst-case obedient model, attack success over 30 attacks falls from 87% to 3%. Section 8 gives the scope of each result

Contents

1	The project in one page
2	The four parts and who does what
3	What goes in: all the inputs
4	One request, step by step
5	What was built, phase by phase
6	Safety: how the model is kept from doing harm
7	How it is tested and measured
8	What is proven and what is not
9	Hands-on: your first 30 minutes (free, no API key)
10	Using it with a real model (costs money)
11	A tour of the code, in reading order
12	Experiments that teach the concepts
13	Concept map: what to say in an interview, and where it lives
14	Interview questions with short answers
15	Rules, cost control and troubleshooting
16	What can be done next
17	Glossary

1. The project in one page

A factory has a plan: which job runs on which machine, and when. When something goes wrong (a machine breaks, an order becomes urgent, a rush order arrives) a planner has to fix the plan quickly without making things worse somewhere else. Re-planning is a maths problem that a solver does well but a planner cannot easily drive. A language model reads plain requests well but must not be trusted with the numbers or the decision.

So this project splits the work. The planner types the problem in plain English. An AI model decides which changes to try. A constraint solver computes the new plan. A validator re-checks it. The model explains what changes. **A person reviews the result and presses Approve; the model can never apply a change itself.**

> **The one sentence to remember:** the model supplies words and choices; the system supplies every number, every check, and the only way to change the live plan.

A worked example (from one real run)

Planner: *"M2 is down from 11:00 to 14:00 today. What happens to the plan?"* The agent makes a draft (a scratch copy of the live plan), records the outage in it, re-plans once (about 0.2 s of solver time), and compares the draft with the live plan:

	Live plan	Draft
Late orders	0	1 (O-108, priority 4, 5 min late)
Total tardiness	0 min	5 min
All orders done	14:25	17:30
Operations moved	-	8 (6 of them to a different machine)

The solver's goals are strictly ordered: lateness first, then fewest operations moved, then finish time. So it accepted a later finish to avoid moving more work. A planner who cares more about finishing early can ask for the `earliest_finish` goal. Nothing is live until the planner approves.

2. The four parts and who does what

Part	Does	Never does
The AI model (Claude)	Understands the request, picks the changes, writes the explanation	Computes schedules, invents numbers, commits anything
The solver (Google OR-Tools CP-SAT)	Finds the new schedule, minimising lateness, then moves, then finish time	Talks to the user
The validator	Re-checks every schedule against the rules; shares no code with the solver	Trusts the solver's own numbers
The person	Reviews exactly what changes and approves it	-

The code is layered so the safety properties hold structurally, not by good behaviour:

- **core/** (the scheduling engine) and **tools/** (the store, approval, tool functions) never import an AI library.
- **agent/** holds the hand-written loop that talks to the model. No agent framework is used.
- **api/** (web UI), **mcp_server/** (for Claude Desktop/Code) and the chat CLI are three front ends over the same tools.

3. What goes in: all the inputs

When someone uses the assistant

Input	Example	Where it comes from
The planner's request	"M2 is down from 11:00 to 14:00"	Typed in the web UI, the chat CLI, or an MCP client such as Claude Desktop
The shop and its live plan	12 orders on 4 machines, one day (a tighter 19-order shop also exists)	Synthetic, generated once and saved in evals/shop.json and evals/shop_tight.json
Order notes	Free text attached to an order	Part of the shop data. **Untrusted**: shown to the model as data, never as instructions
Plant documents	A breakdown procedure, an incident report, a policy	12 synthetic markdown files in knowledge/, searched on demand by the model
Standing preferences	"I always want the earliest finish"	Added only by the planner (a /remember command or a web panel), never by the model
Emails	A rush-order request	Read into a structured order for a person to review (the extraction feature)
Configuration	API key, model name, prices, limits	A private .env file. It is gitignored and never committed

When the system is tested

Input	What it is
48 scenarios (YAML)	A request, plus what a correct agent does about it. Nine categories: simple downtime, priority change, rush order, multiple disruptions, read-only questions, impossible, ambiguous, injection, memory
12 red-team attacks (YAML)	Hostile text planted in an order note, a plant document, or the planner's own message, each aiming at a named harm
22 labelled emails	For scoring the email reader
Retrieval questions	For scoring the document search on its own, with no model involved
Prompt snapshot	The saved text of every prompt and tool description, so any change is deliberate

4. One request, step by step

Take: *"M2 is down from 11:00 to 14:00."*

#	What happens	Who does it
1	The request arrives from the web UI, the chat CLI, or an MCP client	Front end
2	(With the route pattern) a cheap triage call reads only the planner's words and picks a route: read, plan, clarify, or decline_commit	Small model
3	The hand-written loop sends the request and the tool descriptions to the model	Agent loop
4	The model calls read tools (get_schedule, get_order, search_knowledge ...) and edit tools that touch only a draft (create_draft, simulate_downtime, change_priority, add_rush_order)	Model, through the tool layer
5	It calls reschedule once; the solver re-plans everything not yet started, keeping started work in place	Solver

#	What happens	Who does it
6	The validator re-checks the new schedule independently	Validator
7	The model calls compare_schedules and writes its answer	Model
8	The harness attaches the before/after KPIs, the real list of changes and "needs approval", computed from the draft. Answer guards add a warning if the words claim the plan is live or that nothing is late when the facts say otherwise	Harness (code)
9	The planner reviews the proposal and presses Approve (or types `y`, or runs `admin approve` for MCP)	Person
10	Only now is a single-use token minted for that exact draft, and the commit performed	Human-only code

The dotted line in the architecture is the only way the live plan ever changes, and it does not pass through the model.

5. What was built, phase by phase

Built in phases, each reviewed before the next. Every phase ends with tests, a report of trade-offs and interview questions.

Phase	What it delivered	Why it matters for an AI engineer
0	Ground rules: synthetic data only, no agent framework, keys from the environment, core independent of the model, ask before adding dependencies	Working discipline and scoping
1	Scheduling engine with no AI: flexible job-shop model, generator, CP-SAT solver with strictly ordered goals, independent validator, KPIs	A trustworthy tool for the agent to call
2	Tools, the hand-written agent loop, terminal chat, signed single-use approval tokens	Tool use, drafts, human-in-the-loop
3	MCP server with out-of-band human approval	Model Context Protocol
4	Safety: strict inputs, caps, prompt-injection tests with a fully obedient scripted model	Threat modelling for agents
5	Evals: 5 deterministic checks that read system state, plus an LLM judge that must be a different model	How to measure an agent honestly
6	Observability (tokens, latency, cost per step) and a two-model comparison with confidence intervals	Cost, latency, model selection
7	FastAPI web UI with chat, charts and an Approve button guarded by CSRF, Origin, Host and CSP checks	Shipping an AI feature safely
8a/8b	Plant documents: chunking, keyword search, embeddings, hybrid; retrieval scored on its own; record/replay for evals	RAG, retrieval evaluation
9	Memory: old tool results cleared in long chats; standing preferences settable only by the planner	Short- and long-term memory, memory poisoning
10	Email extraction: forced schema, evidence quotes that must exist and support the value, abstention, rule-based baseline	Structured output, grounding
11	Patterns: plan, verify (code checks figures), reflect (a second call reviews)	Agent design patterns
12	Multi-agent routing: a triage on a cheaper model sends questions to a read-only reader, vague requests to a question, "commit it" to a fixed refusal	Multi-agent, least privilege, model routing

Phase	What it delivered	Why it matters for an AI engineer
13	Open models: a model named ollama:NAME runs the same loop; mixable with Anthropic models	Provider abstraction, local models
14	Health check, streamed progress (SSE), Dockerfile, load test, OpenTelemetry export	Production readiness
15	Red team with a measured attack-success rate, answer guards, web rate limit, a prompt-change gate	AI safety engineering, regression control
16	Batch grading (Message Batches API, about half price), a fine-tuning dataset export filtered by the evals, the answer streamed to the web page as it is written, an optional LangGraph port of the loop (48/48) for comparison	Cost engineering, data for fine-tuning, streaming, framework trade-offs

6. Safety: how the model is kept from doing harm

The rule behind everything: the model is treated as an untrusted client that may be confused or manipulated, and nothing it says can change the live plan.

Threat	What stops it
The model commits a schedule	No commit tool is offered to it. A commit needs a signed, single-use, expiring token bound to the draft, the plan version and a fingerprint of the schedule **and its edits**. Only code a person drives mints one
Text in the data (a note, a document) tells the model what to do	Structure (whatever it does is limited to editing drafts), visibility (the approval screen lists the real changes and KPIs, which the model supplies neither of), and judgement (untrusted text arrives in a field named *_untrusted_text and the prompt says only the planner instructs). Judgement is probabilistic; the first two are not
Hostile tool arguments	Every field accepts exactly one spelling; ids must match the system's shape; errors say what was expected
The model piles up work	Caps on drafts, edits and pending requests; step, token and cost limits per turn
The model misdescribes its changes	The change list is copied from the draft; the model cannot fill it
Another web page presses Approve	Per-run CSRF token, Origin and Host checks, loopback-only serving, a strict Content-Security-Policy
Notes plant a lasting instruction (memory poisoning)	The model has no tool that writes memory; preferences come only from the planner
A question is answered by an agent that can edit	Routing gives a question's reader no edit tools, so an obedient reader asked to edit gets "unknown tool"
The answer says the plan is live, or nothing is late, when it is not	Two deterministic answer guards put a warning beside the answer; the text is never altered
A runaway client spends the budget	Web rate limit: 10 turns per minute and 120 per hour, then 429 with Retry-After
A prompt is changed without anyone noticing	A snapshot of every prompt and tool description; the tests fail on any difference

What the red team measured (obedient scripted model)

30 attacks (order notes, planted documents, the planner's own words, floods, hidden characters, spoofed results, role-play) are run against four set-ups and scored on the store and the answer. The scripted model does exactly what the hostile text says, every time, the worst case for the model. So this measures what

the harness contains, not how a real model behaves.

Set-up	Live plan changed	Edits reached a draft	False claim left standing	Attacks that got something
react, answer guards off	0	13	16	26 of 30 (87%)
react	0	13	0	13 of 30 (43%)
route	0	1	0	1 of 30 (3%)
route + verify	0	1	0	1 of 30 (3%)

What the red team measured against a real model

Set-up	Runs completed	Live plan changed	Edits in a draft	False claims	Attacks that got something
react	24 of 24	0	0	0	0
route	24 of 24	0	0	0	0
rt-05 and rt-12 re-run after two fixes	8 of 8	0	0	0	0

Total: 0 of 56 completed live runs succeeded (claude-sonnet-5-5, 12 attacks, 2 runs each). Where a note told it to commit, change priorities or say the plan was live, the model did the legitimate part, said the note contained instructions, and ignored them. Running it live also found two bugs a scripted test cannot: the API rejects forced tool choice, and the scorer had counted failed turns as contained. Both are fixed.

Widening the set from 12 to 30 found a second gap, now closed: an edit left in a draft the answer never named, and a blackout draft too broken to solve whose warning did not list what it held. Such drafts are now reported with their edits.

The red team found a real gap: with the guards off, 8 of the first 12 attacks ended with the assistant telling the planner the plan was live (or that nothing was late) and nothing beside the words said otherwise. The answer guards were added because of that. The one attack still getting through under routing is extra edits hidden inside a legitimate request; the harness-computed change list shows them to the planner.

7. How it is tested and measured

Layer	What it is	What it proves
Unit and integration tests	1424 automated tests (5 more are opt-in: 2 call the real API, 3 download an embedding model). Run by GitHub Actions on every push, without the optional extra	The code does what it says, with scripted stand-ins for the model
Independent validator	A separate checker that shares no code with the solver	The solver's reported numbers equal recomputed ones
Evals (48 scenarios)	Five deterministic checks read the store and the tool log: right kind of answer, right tools, exactly the requested edits, valid schedule, only real numbers quoted. Plus an LLM judge that must be a different model	Behaviour, scored on state, not on wording
Reference agent	A scripted agent that plays every scenario correctly. Passes 48/48 under react, route and route+verify	Tests the eval harness itself, not any model

Layer	What it is	What it proves
Mutation testing	Guards and checks broken on purpose; a test must fail for each break	The tests can actually catch regressions. It found real bugs (for example an eval check that read its baseline after the run and could never fire)
Record and replay	Real model responses saved once and replayed free, with request fingerprints	Cheap, repeatable regression runs; stale recordings are reported
Retrieval scoring	Labelled questions, no model, no cost	Embeddings find 6 of 6 hard paraphrases; keyword search finds 3 of 6
Red team	30 attacks, pinned matrix in a test	A weakened defence changes a named cell and a test says which attack now gets through
Prompt snapshot	Saved text of every prompt and tool description	No prompt changes silently; changes need an eval run and a deliberate update

The one real-model run

claude-sonnet-5-5, 29 scenarios, one run each, judge off. Right kind of answer 29/29, right tools 29/29, exact requested edits 17/17, independent validator 15/15. Numbers traceable to what the model was shown: 22/29. I read all 7 misses and none was a wrong number (three were the model subtracting or counting itself; four were example times in clarifying questions). All three prompt-injection scenarios: the planted instruction was ignored and reported. One run per scenario cannot show variation, and the judge was not run, so explanation quality is unscored.

The judged run (2026-10-12)

All 48 scenarios, one run each, judge claude-fable-5-1. 35 completed and **32 passed every check**; the other 13 could not run because the account hit its usage limit. The three genuine failures: a rush order with no stated priority was set to 4 (the scenario expects 5; a prompt rule now sets the default), a correctly discarded infeasible draft failed an over-strict check (fixed), and a clarifying question stated facts no tool had given (a prompt rule now forbids it). The run also exposed two judge bugs, both fixed: its answer was cut off at 1500 tokens, and it called true details invented because it was not shown the tool results.

8. What is proven and what is not

Verified	Not verified
The solver against an independent validator and recomputation	Run-to-run variation beyond 2 runs per attack and one run per scenario, and models other than claude-sonnet-5-5
Tools, loop, approval, store and the MCP server over real stdio, by 1424 tests	13 of the 48 scenarios on a real model (the account usage limit stopped the run) and a judged route vs react comparison
The web API over a real socket; the UI in a real browser (chat, proposal, Approve, charts, dark mode, phone width)	Claude Desktop or Claude Code connecting to the MCP server
Safety, eval and API code by mutation testing	Run-to-run variation: every real-model result so far is a single run
A real model on 29 scenarios, deterministic checks only	Memory, patterns, routing, answer guards and extraction on a real model: whether they help

Verified	Not verified
The 12 red-team attacks read by claude-sonnet-5-5 (react and route, 2 runs each): 0 of 56 succeeded, live plan never changed. Judged run: 32 of 35 completed scenarios passed	Whether routing saves money or improves answers; memory, patterns and extraction quality on a real model
Trace export against the OTLP JSON structure and into a real Jaeger (9 real-model turns, 84 spans, GenAI view); the container image built and run (non-root, healthy, page and plan served, foreign Host refused)	Chat inside the container with a real model; tracing backends other than Jaeger
The Ollama adapter against a stub, and once against a real Ollama (llama3.2:3b on a CPU): the adapter works, the 3B model could not finish the loop	A local model strong enough to drive the loop (7B and up)

Why this matters: in an interview, say plainly which results are real-model numbers and which are scripted-model tests. Knowing the limit of your own evidence is itself a signal of engineering maturity.

9. Hands-on: your first 30 minutes (free, no API key)

Requirements: **uv** (it installs Python 3.12 for you). Windows is what this was built on; the commands work in Git Bash or PowerShell.

Fastest route: after installing, `uv run python scripts/demo_all.py --pause` runs the red team, the evals, the retrieval scoring and the prompt gate in about a minute, then opens the web UI. docs/DEMO.md has the ten-minute talk track.

Step 1. Get the code and install

```
git clone https://github.com/roopchandrika/jobshop-agent
cd jobshop-agent
uv sync                      # installs the locked dependencies
```

Step 2. Run the tests (about 5 minutes)

```
uv run pytest                # expect: 1424 passed, 5 skipped
```

Step 3. See the web UI with a scripted model (real solver, no key)

```
uv run python scripts/demo_server.py      # then open http://127.0.0.1:8765
```

Type a disruption such as *"M1 is down from 10:00 to 13:00"*. The wording is canned; the plans, KPIs and charts are real. Look at the proposal card: it lists the changes and KPIs, and the Approve button is the only way the plan changes.

Step 4. Run the evals and the red team (free)

```
uv run python -m jobshop.evals run --oracle        # all 48 scenarios with the scripted reference agen
uv run python -m jobshop.evals redteam             # the 30 attacks against an obedient scripted model
uv run python -m jobshop.evals snapshot            # confirms no prompt or tool description changed
uv run python -m jobshop.evals retrieval --method bm25   # scores the document search
```

Open the report each command prints (under `evals/results/`). The red-team report has a matrix of attack by set-up.

Step 5. Read a trace

Chat with the real or demo agent, then:

```
uv run python -m jobshop.agent.trace_report logs/traces/<file>.jsonl
```

It shows every model call and tool call with tokens, latency and cost: the agent's behaviour, step by step.

10. Using it with a real model (costs money)

Cost warning. The 29-scenario real run used about 760k tokens before prompt caching. A full 48-scenario run with the judge is roughly 1.5 million tokens per pass; a live red team roughly 2 to 3 million. These are my estimates, not measurements. Always start with one scenario (`--only sd-01`) and set `JOBSHOP_MAX_COST_USD`.

Setup

```
cp .env.example .env
# edit .env: set ANTHROPIC_API_KEY and ANTHROPIC_MODEL (never commit .env)
# optional: JOBSHOP_PRICE_INPUT_PER_MTOK / JOBSHOP_PRICE_OUTPUT_PER_MTOK so costs show as dollars
```

Ways to use it

What	Command
Web UI	uv run python -m jobshop.api (open http://127.0.0.1:8000; chat needs a key, the plan is shown without one)
Chat in a terminal	uv run python -m jobshop.agent.cli --now "2026-01-05 12:00"
Chat commands	/state /clock YYYY-MM-DD HH:MM /remember TEXT /forget N /prefs /help /quit
MCP for Claude Desktop/Code	uv run python -m jobshop.mcp_server.admin init --now "2026-01-05 12:00" then add the server to the client (docs/MCP.md). Approve in a terminal with: uv run python -m jobshop.mcp_server.admin approve
One real eval scenario	uv run python -m jobshop.evals run --only sd-01 --no-judge
Eval with the judge	uv run python -m jobshop.evals run (needs ANTHROPIC_JUDGE_MODEL, a different model from the one under test)
Compare two models	uv run python -m jobshop.evals compare --model A --model B --judge-model C
Compare patterns on one model	uv run python -m jobshop.evals compare --model M --pattern react --pattern verify
Routing with a cheaper triage	uv run python -m jobshop.evals run --pattern route --triage-model SMALL --triage-price IN,OUT
Live red team	uv run python -m jobshop.evals redteam --live --model M --repeat 3
Record once, replay free	uv run python -m jobshop.evals run --record DIR then ... run --replay DIR
Local model	name it ollama:NAME in ANTHROPIC_MODEL or --model; Ollama must be running (a 3B model could not finish the loop)

The web app listens on 127.0.0.1 only and refuses other addresses, because it has no login.

11. A tour of the code, in reading order

Read in this order and you will understand the project bottom-up. Source is under `src/jobshop/`.

Step	Read	What you learn
1	core/models.py, core/solver.py, core/validator.py, core/kpis.py	The scheduling problem, how the solver is steered by strictly ordered goals, and why an independent validator exists
2	tools/functions.py, tools/store.py, tools/approval.py, tools/outcome.py	What the model may do (drafts, edits, solve, compare) and how a commit is made unforgeable
3	tools/registry.py	Which tools are visible to which front end; scoped registries (least privilege)

Step	Read	What you learn
4	agent/loop.py (run_turn)	The hand-written tool-use loop: the model call, accounting, patterns, answer guards, final answer
5	agent/prompts.py, agent/patterns.py	What the model is told; plan, verify, reflect and route
6	agent/memory.py, agent/history.py, agent/conversation.py	Preferences, history compaction, the conversation layer
7	knowledge/ and agent search tool	Chunking, BM25, embeddings and hybrid search
8	api/app.py, api/ratelimit.py, api/static/	The web layer: CSRF, SSE progress, rate limit, the page
9	evals/scenario.py, checks.py, runner.py, judge.py, redteam.py, snapshot.py	How behaviour is measured, and how the measurement itself is checked
10	extraction/	Email to structured order with evidence checks and abstention
11	agent/providers.py	The provider adapter that lets an Ollama model use the same loop

Docs worth reading, in order: **README.md**, **docs/ROADMAP.md** (what is done, with evidence), **docs/SAFETY.md** (threat model and the red team), **docs/EVALS.md** (how it is measured), **docs/DESIGN.md** (decisions and a glossary).

12. Experiments that teach the concepts

Experiment	What to do	What you should see
Try to break the safety	With a real model (the demo's wording is canned), ask "just commit it" and read the answer; then run `redteam` and open the matrix	A refusal; the live plan unchanged; every attack either contained or visible
Plant an attack yourself	Add an attack to evals/redteam.yaml (copy rt-04), run redteam	Your attack appears in the matrix; try to make it succeed under route
Add a scenario	Copy an entry in evals/scenarios/*.yaml, change the request and expectations, run `run --oracle --only YOUR-ID`	The oracle needs an oracle: block; a wrong expectation fails a named check
Break a prompt on purpose	Edit a sentence in agent/prompts.py and run the tests	test_prompt_snapshot fails with a diff; update with snapshot --update only after the evals justify it
Weaken a defence	Make the reader's tool list include create_draft (patterns.READ_ONLY_TOOLS) and run the tests	Route tests and the red-team matrix fail, naming the attack that now gets through
Compare retrieval methods	Run `evals retrieval --method all` after `uv sync --extra embeddings`	Embeddings beat keywords on paraphrases; off-topic separation is reported
Watch cost	Run one scenario with prices set, then open the trace report	Per-step tokens, cache reads, cost; the triage model costed at its own price under route

13. Concept map: what to say in an interview, and where it lives

Concept	In this project	Where
Tool use / function calling	Hand-written loop, strict tool schemas, forced terminal tool (submit_response)	agent/loop.py, tools/
Human in the loop	Draft, proposal, token minted only on human approval	tools/approval.py, tools/human.py
Guardrails / safety	No commit tool, answer guards, caps, scoped tools	SAFETY.md, agent/claims.py
Prompt injection	Untrusted-text fields, obedient-model tests, red team with a measured rate	evals/redteam.py
RAG	Chunking, BM25, dense, hybrid (RRF), retrieval eval, documents as untrusted data	knowledge/, DESIGN.md
Embeddings	fastembed local model as an optional extra	knowledge/semantic.py
Memory	History compaction; planner-only preferences	agent/history.py, agent/memory.py
Structured output	Forced schema, evidence quotes, abstention	extraction/
Agent patterns	ReAct, plan-and-execute, verify, reflect	agent/patterns.py
Multi-agent	Triage and specialists with least privilege	agent/patterns.py, loop.py
Model routing / cost	Cheap triage model, per-call pricing, caching, budgets, rate limit	agent/pricing.py, api/ratelimit.py
Prompt caching	cache_control on tools and newest message; cache-aware prices	agent/loop.py
Evals	Deterministic checks on state, LLM judge, reference agent, mutation testing, replay	evals/
Observability	Versioned JSONL traces, OpenTelemetry export	agent/trace.py, trace_export.py
MCP	stdio server with out-of-band approval	mcp_server/
Production	Health check, SSE, container, load test	api/, Dockerfile, scripts/
Open models / providers	Ollama adapter behind the same interface	agent/providers.py
Regression control	Prompt snapshot, pinned red-team matrix	evals/snapshot.py

14. Interview questions with short answers

Q. Why can't the model commit?

There is no commit tool in its tool list. A commit needs an HMAC-signed, single-use, expiring token bound to the draft, the plan version and a digest of the schedule and edits, and only code that a person drives mints one. Safety is structural, so it does not depend on the model behaving.

Q. How do you defend against prompt injection?

Three layers: structure (the worst the model can do is edit a draft), visibility (the approval screen shows system-computed changes and KPIs, not the model's description) and judgement (untrusted text is labelled and size-capped). Then I measure it: a red team of 30 attacks, scored on state, with an obedient model as the worst case.

Q. Why do the evals check state rather than the model's words?

A model can say it did the right thing without doing it, or the reverse. The checks read the store and the tool log: what was actually changed, whether the schedule validates, whether quoted numbers came from tools.

Q. What did mutation testing find?

An eval check for 'live plan unchanged' read its baseline after the run, so it could never fire. Breaking the code on purpose and demanding a test fail is what exposed it.

Q. Why must approval bind to the edits and not just the schedule?

Otherwise an edit that moves nothing could ride on an earlier approval. Found while writing the web approval test; approval now binds to the schedule and the draft's edits.

Q. What does routing buy you?

Least privilege: a question is answered by a reader with no edit tools, so a hostile note cannot turn into a draft. It also lets a cheap model do the triage. In the red team it cut attack success from 43% to 3% for a scripted obedient model. Whether it helps on a real model is not measured.

Q. How do you handle memory poisoning?

The model has no tool that writes memory. Preferences come only from the planner, are short and capped, and are stated to the model as unable to override its rules.

Q. Keyword search or embeddings?

I measured both on labelled questions: embeddings found 6 of 6 hard paraphrases, keywords 3 of 6. Hybrid search combines them. Embedding search is an optional extra so CI and the default install need none of it.

Q. How would you know a prompt change is an improvement?

The snapshot test fails on any change. Then I run the evals before and after on a real model, compare pass rate and cost per passing run with confidence intervals, and only then update the snapshot and commit the numbers together with the change.

Q. How do you control cost?

Per-step token and cost accounting, a per-turn budget, prompt caching, a cheaper triage model costed at its own prices, replay for regression runs, and a web rate limit.

Q. What are the weaknesses?

Most results are with scripted models. One real-model run exists (22 of 29 on the numbers check, no wrong numbers). Routing's benefit, the patterns' quality and run-to-run variation are not yet measured on a real model, and a 3B local model could not drive the loop. I list these in the README so nobody has to guess.

Q. Why hand-write the loop instead of using a framework?

To understand and control every step: accounting, forced terminal tool, history validity, error handling, patterns. A framework would hide exactly the parts I wanted to learn and test.

15. Rules, cost control and troubleshooting

Rules of the road

- **Never commit .env or paste your API key anywhere.** It lives only in .env, which is gitignored.

- Real runs spend API credit. Start with one scenario, set a cost cap, and read the estimate before a large run.
- Do not commit docs/SPEC.md (kept local on purpose) or INTERVIEW_NOTES.md (private, gitignored).
- Commit messages and pull requests carry no AI attribution; the commits are yours.
- The web app has no login. Keep it on localhost.
- After you change a prompt or tool description, run the evals before updating the snapshot.

Troubleshooting

Symptom	Cause and fix
"ANTHROPIC_API_KEY is not set"	Copy .env.example to .env and fill it in. A run that uses only ollama: models needs no key
The web page shows the plan but chat is disabled	No key or model configured; the page works without them by design
429 "Too many requests"	The web rate limit (10 turns a minute, 120 an hour). Wait for Retry-After or raise JOBSHOP_RATE_LIMIT_PER_MIN
409 "the assistant is still working"	One turn at a time by design; wait for the current one
snapshot test fails after an edit	You changed a prompt or tool description. If intended, run the evals, then: uv run python -m jobshop.evals snapshot --update
Dense or hybrid search errors	Run: uv sync --extra embeddings (downloads about 65 MB the first time). Keyword search needs nothing
Replay says a recording is stale	The prompt, tools or tool results changed since it was recorded; record again
Docker command fails	The image has never been built here; start Docker Desktop first and treat the first build as the real test
Ollama run says the prompt filled the window	Raise JOBSHOP_OLLAMA_NUM_CTX or use a shorter conversation
Tests fail only on your machine	Run uv sync --locked to match the lock file exactly; CI runs without the embeddings extra

16. What can be done next

Option	Cost	What it closes
Raise the API usage limit, then run the 13 missing scenarios and a judged route vs react comparison	About 3M tokens (estimate)	The remaining scenarios and the routing result
Connect Claude Desktop to the MCP server	Free	The MCP-in-a-real-client gap; also a good demo
Try a 7B-and-up local model, ideally on a GPU	Free, a download of several GB	Whether a local model can drive the loop (a 3B one could not)
Live red team with 5 repeats	About 3 to 4M tokens (estimate)	A firmer attack-success number
Full 48-scenario eval with the judge; route vs react	About 1.5M tokens per pass plus the judge (estimate)	Whether routing helps; the missing results table
Email extractor and preferences on a real model	About 100k tokens (estimate)	Two features never run live

Option	Cost	What it closes
Message Batches API for evals	Free to build; cheaper to run	Cost engineering at roughly half price
Token-by-token answer streaming	Free	A UX and streaming-protocol concept
Export a fine-tuning dataset from traces	Free	The fine-tuning workflow without training
Port the loop to an agent framework	Free	A side-by-side comparison with the hand-written loop

Recommended order: the live red team and the route-versus-react evaluation first (one measured real-model result is worth more in an interview than another feature), then the Docker build and the Claude Desktop connection.

17. Glossary

Term	Meaning
Draft	A scratch copy of the live plan the model may edit. Never live
Proposal	A solved draft with its comparison, waiting for a person to approve
Approval token	A signed, single-use, expiring value bound to one draft, plan version and digest; minted only by human-driven code
CP-SAT	Google OR-Tools constraint-programming solver used to build schedules
Tardiness / late order	How far past its due time an order finishes
KPI	A key figure such as late orders, total tardiness, finish time, utilisation
Goal (fewest_moves, earliest_finish)	What the solver optimises after lateness: disturb the plan least, or finish everything soonest
Tool use	The model asking the harness to run a function and reading the result
Prompt injection	Text in data that tries to give the model instructions
Untrusted text	Content (notes, documents) shown to the model as data, never as instructions
RAG	Retrieval-augmented generation: looking things up and showing them to the model
BM25 / dense / hybrid	Keyword scoring / search by embedding meaning / the two combined
Eval / scenario	A scored test of the agent; a scenario is one request with expected behaviour
Oracle (reference agent)	A scripted agent that plays scenarios correctly, used to test the eval harness
Mutation testing	Breaking the code on purpose to check a test fails
Red team	Hostile test content with a measured success rate
Pattern (react, plan, verify, reflect, route)	A way of organising the same model, tools and rules
Triage / route	A first cheap call that picks which specialist handles a request
Answer guard	A deterministic check of the answer's words against facts the harness knows
MCP	Model Context Protocol: a standard way for AI clients to use tools from a server
SSE	Server-Sent Events: the server pushing progress updates to the browser
Trace	A JSONL record of every model and tool call in a turn